

Neural Shape Diameter Function for Efficient Mesh Segmentation

Bruno Roy
bruno.roy@autodesk.com
Autodesk Research
Montreal, Canada

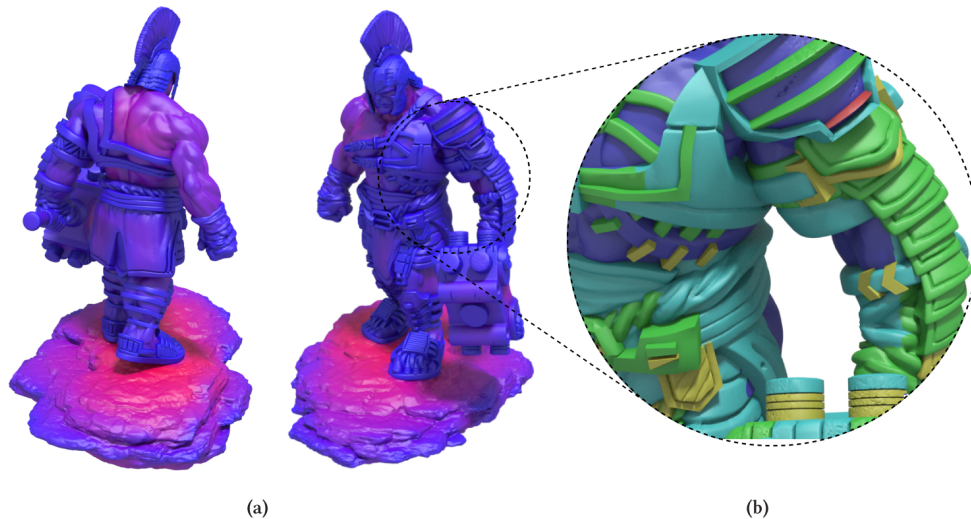


Figure 1: (a) With our neural SDF, we are capable of generating consistent SDF values across various high-resolution man-made and organic shapes at a fraction of the cost of its GPU implementation (red: narrow diameter and blue: wide diameter). (b) The approximated SDF values are then used with a state-of-the-art graph-cut algorithm to partition the input mesh into meaningful clusters of elements.

ACM Reference Format:

Bruno Roy. 2023. Neural Shape Diameter Function for Efficient Mesh Segmentation. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH '23 Posters)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3588028.3603652>

1 INTRODUCTION

Over the last two decades, mesh segmentation has been used for various applications in computer graphics. This task has proven to benefit many applications in 3D shape analysis such as texture mapping [Sander et al. 2003], 3D shape modeling [Ji et al. 2006], 3D shape retrieval [Ferreira et al. 2010], multi-resolution and mesh compression [Maglo et al. 2011], and animation [Yuan et al. 2016]. We refer the reader to this survey for further details [Rodrigues et al. 2018] on part-based mesh segmentation. Historically, the traditional problem of mesh segmentation has been approached in

several ways such as by region growth, clustering, implicit boundaries, and volume-based methods (e.g., shape diameter function). The shape diameter function (SDF) algorithm is at the intersection of using implicit boundaries and volume-based methods. The intuition behind SDF is to produce a diameter measure throughout vertex neighborhoods of the mesh. The resulting measure (i.e., SDF values) relates to the medial axis transform and provides a volume correspondence of the shape at any given point on the surface. These per-vertex measures are then used as a threshold for the graph-cut algorithm. Even though the SDF method offers high-quality and consistent results, it is hardly scalable to dense meshes (i.e., millions of polygons), even when implemented on GPU [Chen et al. 2018].

More recently, neural networks have also been widely used for the segmentation problem on images [Lai 2015], point clouds [Qi et al. 2017], and meshes [Hanocka et al. 2019]. Several interesting approaches were proposed lately to tackle the mesh segmentation problem using deep neural network algorithms such as using convolution operators on edges [Hanocka et al. 2019], converting 3D shapes into voxel-based representations [Graham et al. 2018], and leveraging local features of point clouds [Qi et al. 2017]. Nevertheless, these state-of-the-art data-driven methods remain hardly generalizable and often require dividing the learned model into several specific classes of objects to avoid overfitting. In contrast to

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Posters, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0152-8/23/08.

<https://doi.org/10.1145/3588028.3603652>

these methods and [Kovacic et al. 2010], we avoid the computationally expensive part by using predicted SDF values $\tilde{\lambda}$ to generate the final mesh segmentation at the fraction of the cost of [Chen et al. 2018]. Moreover, learning a mapping function as opposed to directly learning to classify mesh elements [Kalogerakis et al. 2010] (e.g., to cluster vertices) makes our approach more robust when used in unknown settings.

2 APPROACH

As a formal definition, the traditional mesh segmentation task is described as follows: given a mesh $\mathbf{M}$, and $\mathbf{E}$ the set of mesh elements (i.e., vertices, edges, or faces). The segmentation $\mathbf{S}$ of mesh $\mathbf{M}$ is a set of sub-meshes $\mathbf{S} = \{M_0, \dots, M_{n-1}\}$, where M_i is defined as a subset of elements $e \in \mathbf{E}$. In the traditional SDF method, the set of sub-meshes $\mathbf{S}$ is obtained by subdividing a graph using the diameter measures λ_i from per-vertex neighborhoods as thresholds for the graph-cut algorithm. As we aim to estimate these SDF values through a graph neural network, the problem of finding the sub-meshes M_i can be expressed as partitioning $\mathbf{S}$ such that the constraint criteria $\{C_0, \dots, C_n\}$ are minimized. The predicted SDF values $\tilde{\lambda}_i$ are then used as threshold criteria to a graph-cut algorithm to generate the sub-meshes $\mathbf{S} = \{M_0, \dots, M_{n-1}\}$. The constraint criteria $\{C_0, \dots, C_n\}$ are defined as two terms: the similarities between the reference SDF values λ_i and the predicted SDF values $\tilde{\lambda}_i$, and the neighborhood densities ρ_i of vertices. We express the L_2 similarities between the reference SDF values and the predicted ones as follows:

$$L_\Lambda = \frac{1}{n_j} \sum_i^{n_j} \alpha \|\lambda_i - \tilde{\lambda}_i\|_2, \quad (1)$$

where n_j is the number of downsampled surface vertices (i.e., used to query the neighborhoods). Along with the input vertices, we provide our network with an additional term to weigh in the local mesh density for adaptive resolution meshes. We use the Poisson disk sampling method at the surface of meshes to select the downsampled vertices and compute their neighborhood density. The resulting densities ρ_i are then used as a scaling factor during the messaging stage within our network to properly propagate the attributes in the current neighborhood.

Once the predicted SDF values $\tilde{\lambda}$ are obtained, we use them as inputs to a fast k -way graph-cut algorithm to offer an efficient and flexible way for mesh segmentation. Similarly to [Shapira et al. 2008], our partitioning algorithm is composed of two steps. The first uses soft-clustering of the mesh elements (faces) to compute k clusters based on their SDF values, and the second finds the actual partitioning using k -way graph-cut to include local mesh geometric properties.

Our network architecture is essentially based on the Encode-Message-Decode model. The network is composed of two hidden MLP layers on both Encoder and Messenger with an output size of 128. The resulting Decoder output size matches the downsampled vertices provided as input during training and inference. Our model is trained and loaded for inference using an A6000 GPU with the Adam optimizer for 5M training steps with an exponential learning rate decay from 10^{-3} to 10^{-5} when passing the threshold of 3M steps. In our network, we handle varying resolutions using two

mechanisms: downsampling the input mesh while maintaining full-resolution neighborhoods and using the neighborhood densities ρ_i for each node m^i as scaling factors when updating the current nodes during the messaging stage. The idea to remain resolution-agnostic is to pass the full-resolution neighborhoods along the downsampled mesh. That way, we can query the vertex neighborhoods by solely using the downsampled mesh vertices to compute the SDF values $\tilde{\lambda}$. The density values are used to scale the contributions of the neighbors through the edges e^i during the message-passing step in our network. The dataset used to train our network is generated using the *Autodesk Character Generator*© tool producing multiple variants of the same base mesh by applying blend shape operations on it (roughly 10k augmented models). Moreover, we have built an additional custom dataset using TurboSquid (TS) assets to evaluate our approach with production-ready content. Our approach generates a fairly low error rate on SDF values compared to the baseline (i.e., smaller than 1% on average).

3 CONCLUSION

We have presented a neural shape diameter function approach allowing us to considerably accelerate high-quality mesh segmentation while remaining adaptive and controllable. For example, our approach generated a segmentation of the *Gladiator Hulk* (source: desirefx.me) in Fig. 1b in about 35 seconds as opposed to a bit less than 9 minutes with [Chen et al. 2018]. For more details, please consult our full paper on arXiv.

REFERENCES

- Shuangmin Chen, Taijun Liu, Zhenyu Shu, Shiqing Xin, Ying He, and Changhe Tu. 2018. Fast and robust shape diameter function. *Plos one* 13, 1 (2018), e0190666.
- Alfredo Ferreira, Simone Marini, Marco Attene, Manuel J Fonseca, Michela Spagnuolo, Joaquim A Jorge, and Bianca Falcidieno. 2010. Thesaurus-based 3D object retrieval with part-in-whole matching. *International Journal of Computer Vision* 89, 2 (2010), 327–347.
- Benjamin Graham, Martin Engelcke, and Laurens Van Der Maaten. 2018. 3d semantic segmentation with submanifold sparse convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 9224–9232.
- Rana Hanocka, Amir Hertz, Noa Fish, Raja Giryes, Shachar Fleishman, and Daniel Cohen-Or. 2019. Meshcnn: a network with an edge. *ACM Transactions on Graphics (TOG)* 38, 4 (2019), 1–12.
- Zhongping Ji, Ligang Liu, Zhonggui Chen, and Guojin Wang. 2006. Easy mesh cutting. In *Computer Graphics Forum*, Vol. 25. Wiley Online Library, 283–291.
- Evangelos Kalogerakis, Aaron Hertzmann, and Karan Singh. 2010. t. In *ACM SIGGRAPH 2010 papers*. 1–12.
- Maurizio Kovacic, Fabio Guggeri, Stefano Marras, and Riccardo Scateni. 2010. Fast approximation of the shape diameter function. In *Proceedings Workshop on Computer Graphics, Computer Vision and Mathematics (GraVisMa)*, Vol. 5.
- Matthew Lai. 2015. Deep learning for medical image segmentation. *arXiv preprint arXiv:1505.02000* (2015).
- Adrien Maglo, Ian Grimstead, and Céline Hudelot. 2011. Cluster-based random accessible and progressive lossless compression of colored triangular meshes for interactive visualization. In *Proceedings of computer graphics international*.
- Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. 2017. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 652–660.
- Rui SV Rodrigues, José FM Morgado, and Abel JP Gomes. 2018. Part-based mesh segmentation: a survey. In *Computer Graphics Forum*, Vol. 37. Wiley Online Library, 235–274.
- Pedro V Sander, Zoë J Wood, Steven Gortler, John Snyder, and Hugues Hoppe. 2003. Multi-chart geometry images. (2003).
- Lior Shapira, Ariel Shamir, and Daniel Cohen-Or. 2008. Consistent mesh partitioning and skeletonisation using the shape diameter function. *The Visual Computer* 24, 4 (2008), 249–259.
- Qing Yuan, Guigang Li, Kai Xu, Xudong Chen, and Hui Huang. 2016. Space-time co-segmentation of articulated point cloud sequences. In *Computer Graphics Forum*, Vol. 35. Wiley Online Library, 419–429.